Projeto: Resolvendo o ARC-AGI-1 com LLMs

IA002 · Explorando o Potencial de Agentes de IA Generativa

André Grahl Pereira

1 Desafio

O ARC-AGI-1 reúne tarefas nas quais alguns pares de grades de entrada e saída apresentam uma transformação. Um solucionador (*solver*) deve inferir essa transformação e produzir a saída correspondente a uma nova entrada. Cada tarefa pode exigir uma regra diferente, e uma solução somente é considerada correta quando toda a grade produzida coincide com a resposta esperada.

Neste projeto, usamos um modelo de linguagem (LLM) para resolver tarefas do ARC-AGI-1 de duas formas:

- **Fim a fim:** o LLM recebe os exemplos e escreve diretamente a grade de resposta.
- **Geração de código:** o LLM escreve um programa em Python, e o computador executa esse programa para produzir a resposta.

Tarefas exemplo: 3aa6fb7a, 44d8ac46 e 44f52bb0.

2 Organização

- O projeto é **individual**.
- Há **três níveis** de dificuldade. Cada aluno escolhe **um** nível e entrega apenas esse nível.
- **Completar o nível 1 é suficiente para a aprovação.**
- O nível 1 é independente: ele usa um notebook pronto, e quem faz os níveis 2 ou 3 não precisa fazê-lo. Os níveis 2 e 3 formam uma sequência: o nível 3 inclui o nível 2.
- Nos níveis 2 e 3, o solver deve ser construído com a ajuda de um agente de codificação. Qualquer agente pode ser usado, por exemplo Claude Code, Codex, Antigravity, Copilot ou OpenCode.
- A entrega é feita na **tarefa do Moodle**.

3 Configuração Comum

- **Modelo:** Gemini 3.5 Flash Lite (`gemini-3.5-flash-lite`), pela API do Google.
- **Chave da API:** gratuita, criada em `https://aistudio.google.com/api-keys`. Ela deve ficar nos *Secrets* do Colab (nível 1) ou num arquivo `.env` (níveis 2 e 3), e nunca deve aparecer no código ou no relatório.
- **Pausa entre chamadas:** espere 1 segundo entre duas chamadas à API, para não exceder o limite da chave gratuita.
- **Tarefa resolvida:** todas as grades de teste produzidas são exatamente iguais às respostas esperadas.

- **Tokens:** registre, para cada tarefa, os tokens de *entrada* (o tamanho do *prompt*), de *raciocínio* (os tokens que o modelo gera antes da resposta final) e de *saída* (o tamanho da resposta).
- **Tempo:** registre, para cada tarefa, o tempo de execução em segundos, incluindo as chamadas à API e a execução do código gerado.

4 Materiais Fornecidos

- `arc_solver_nivel1.ipynb`: o notebook do nível 1, pronto para o Google Colab.
- `tarefas.zip`: as 20 tarefas do projeto em JSON, usadas em todos os níveis. A pasta `training/` tem 10 tarefas do conjunto de treino, e a pasta `evaluation/` tem 10 tarefas do conjunto de avaliação. A lista `tarefas.txt` traz o caminho de cada arquivo.
- `template.zip`: o modelo de relatório no formato IJCAI, para os níveis 2 e 3. Ele pode ser aberto no Overleaf (*New Project* e *Upload Project*).

5 Nível 1: Notebook

O que fazer.

1. Abra `https://colab.research.google.com`, escolha *Arquivo* e *Fazer upload de notebook*, e envie o arquivo `arc_solver_nivel1.ipynb`.
2. Siga as instruções do próprio notebook: crie a chave da API, guarde-a nos *Secrets* do Colab e rode as células na ordem.
3. O notebook resolve as 20 tarefas em quatro configurações: fim a fim ou geração de código, cada uma com raciocínio mínimo e alto.
4. Baixe o arquivo `resultados.csv` gerado pelo notebook.
5. Baixe o notebook executado, com as saídas visíveis, pelo menu *Arquivo*, *Fazer download*, *Fazer download do .ipynb*, e renomeie o arquivo para `nivel1.ipynb`.
6. Escreva o relatório com a ajuda do ChatGPT, a partir do arquivo `resultados.csv`, e revise os números.

O que medir. Para cada uma das quatro configurações: tarefas resolvidas (de 20), total de tokens de entrada, raciocínio e saída, e tempo.

O que entregar. Os arquivos descritos na Seção 9: o notebook executado, o arquivo `resultados.csv` e um relatório de **até duas páginas** em PDF, em formato livre (Word, Google Docs ou outro), com:

1. **Identificação:** o nome completo do aluno, no topo da página.
2. **Objetivo:** o que foi comparado.
3. **Configuração:** modelo, as quatro configurações e as 20 tarefas.
4. **Resultados:** a tabela por configuração.
5. **Exemplos:** uma tarefa resolvida e uma não resolvida, com uma explicação curta.
6. **Conclusão:** qual configuração funcionou melhor e a que custo em tokens.
7. **Uso de IA:** uma frase sobre como o ChatGPT foi usado.

6 Nível 2: Solver com Agente de Codificação

O que fazer. Usando um agente de codificação, construa em Python um solver que:

1. leia a lista `tarefas.txt` e resolva as 20 tarefas;
2. resolva cada tarefa **fim a fim**, pedindo ao LLM a grade de saída;
3. resolva cada tarefa por **geração de código**, pedindo ao LLM uma função `transform(grid)` em Python, que é executada com tempo limite nos exemplos de treino e na entrada de teste;
4. use o raciocínio **alto** nas duas abordagens;
5. salve, para cada abordagem, um resumo com o resultado de cada tarefa, os tokens e o tempo.

O que medir. Para cada abordagem: tarefas resolvidas (de 20), total de tokens de entrada, raciocínio e saída, e tempo total. Na geração de código, registre também em quantas tarefas o programa acertou todos os exemplos de treino.

O que entregar. A pasta descrita na Seção 9, com o relatório de **até duas páginas** no template IJCAI, contendo o resumo, a introdução, o método, a tabela de resultados, um exemplo de acerto e um de erro, e a conclusão.

7 Nível 3: Correção com Exemplos de Treino

O que fazer. Faça o nível 2 e acrescente um laço à geração de código: enquanto o programa gerado não acertar todos os exemplos de treino, chame o LLM de novo, enviando o programa anterior e o erro (a exceção ou a diferença entre a saída obtida e a esperada). Pare quando o programa acertar o treino ou depois de **5 tentativas**. Use o último programa gerado para responder ao teste.

O que medir. Além das medidas do nível 2: tarefas resolvidas com o laço (de 20), número médio de tentativas e número de tarefas em que o programa acertou o treino mas errou o teste. Compare com a geração de código sem o laço.

O que entregar. O mesmo que no nível 2, com o laço descrito no método e as novas medidas no relatório.

8 Observação sobre o Modelo

Com o Gemini 3.5 Flash Lite, as diferenças entre as abordagens dos níveis 2 e 3 (geração de código e correção com os exemplos de treino) podem ser pequenas. O modelo já é grande e resolve boa parte das tarefas diretamente. Essas técnicas tendem a produzir ganhos maiores em modelos menores.

Não é obrigatório, mas opcional, observar esse efeito repetindo os experimentos com o Gemma 4 31B, disponível na mesma API, ou com um modelo ainda menor. Modelos menores podem rodar no ambiente da competição do ARC Prize, que não permite o uso de APIs externas.

9 Entrega

Envie um único arquivo `nome_sobrenome.zip` na tarefa do Moodle. No topo do relatório, indique o seu nome completo e o nível entregue.

Nível 1:

```
nome_sobrenome.zip
  nivel1.ipynb        o notebook executado, com as saídas visíveis
  resultados.csv      gerado pelo notebook
  relatorio.pdf       até duas páginas
```

Níveis 2 e 3:

```
nome_sobrenome.zip
  nome_sobrenome/
    solver/           o código do solver
      README.md       o comando para rodar o solver
    resultados/       um resumo por abordagem (resolvidas, tokens e tempo)
    relatorio.pdf     até duas páginas, template IJCAI
    relatorio.tex     o código-fonte do relatório
```

10 Checklist

Nível	Itens
1	Arquivos presentes. O notebook mostra as saídas das quatro configurações. A tabela do relatório coincide com `resultados.csv`. O relatório tem até duas páginas, o nome do aluno e as seis seções pedidas.
2	Arquivos presentes. O `README.md` explica como rodar. Os resumos em `resultados/` cobrem as 20 tarefas nas duas abordagens. O relatório traz resolvidas, tokens e tempo das duas abordagens, coincidindo com `resultados/`. O relatório tem até duas páginas e indica o nível.
3	Tudo do nível 2. O laço de correção está no código e no método. O relatório traz resolvidas com o laço, a média de tentativas e os casos que acertam o treino e erram o teste.

11 Sugestão Adicional: Votação entre Programas

Esta sugestão não é obrigatória e não faz parte da entrega. Ela fica de fora dos níveis porque pode exigir até 500 chamadas à API, o limite diário da chave gratuita, e não caberia no dia da oficina.

Quem tiver feito o nível 3 pode, para cada tarefa, rodar o laço de correção **5 vezes**, de forma independente, obtendo até 5 programas. Aplique os programas à entrada de teste e use como resposta a **saída mais frequente**. Em caso de empate, use a saída do primeiro programa que acertou todos os exemplos de treino. Compare as tarefas resolvidas, os tokens e o tempo com os do nível 3, que usa um único programa.

12 Leituras e Recursos

- **ARC Prize 2024: Technical Report.** Leitura principal sobre as soluções de 2024: `https://arcprize.org/media/arc-prize-2024-technical-report.pdf`
- **fchollet/ARC-AGI.** Dados e interface para exploração manual: `https://github.com/fchollet/ARC-AGI`
- **ARC-AGI-1.** Apresentação geral do benchmark: `https://arcprize.org/arc-agi/1`
- **ARC Prize Guide.** Formato dos dados, desenvolvimento e famílias de abordagens: `https://arcprize.org/guide/1`
- **On the Measure of Intelligence.** Motivação e concepção do benchmark: `https://arxiv.org/abs/1911.01547`